

case, where possible, you should specify the specific class or classes that you want to import, rather than importing the whole file. Here's the syntax for importing particular classes:

---

```
from <ExternalFile> import <ClassName1>, <ClassName2>, ...
```

---

There are two benefits to importing class code:

1. The module is reusable, so if we want to use *Account.py* in some other project, we just need to make a copy of the file and place it into that project's folder. Reusing code in this way is a staple of object-oriented programming.
2. If your class code is included in the main program, every time you run the program, Python compiles all the code in your class (translates it into a lower-level language that is more easily runnable on your computer), even if you have not made any changes to the class.

However, when you run your main program with your class code imported, Python optimizes the compile step without you having to do anything. It creates a folder named `__pycache__` in the project folder, then compiles the code in your class file and saves the compiled code in the `__pycache__` folder with a variant of the original Python filename. For example, for the *Account.py* file, Python will create a file using the name *Account.cpython-39.pyc* (or similar, based on the version of Python you are using). The *.pyc* extension stands for *Python Compiled*. Python only recompiles your class file if the source of the class file changes. If the source of your *Account.py* has not changed, Python knows it doesn't need to recompile it and can more efficiently use the *.pyc* version of the file instead.

## Creating Some Test Code

We'll test our new class with four main programs. The first will create `Account` objects using separately named variables. The second stores the objects in a list, while the third stores the account numbers and objects in a dictionary. Finally, the fourth version will split the functionality so we have